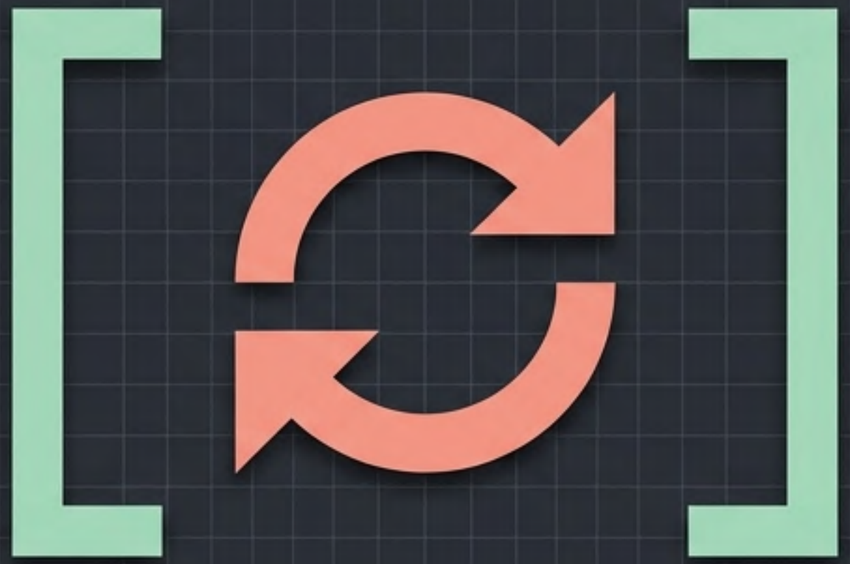
Arrays y Bucles en JavaScript

MANUAL (Array)

```javascript
const datos = ["manzana", "naranja", "uva"];

console.log(datos[0]);
console.log(datos[1]);
```



AUTOMÁTICO (Bucle)

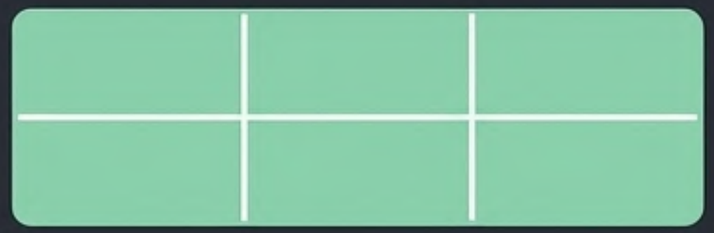
```javascript
for (let i = 0; i < datos.length; i++) {
   console.log("Procesando: " + datos[i]);
}
```

De lo manual a lo automático

NotebookLM

El problema de los datos dispersos

```
let fruta1 = "Manzana";
let fruta2 = "Naranja";
let fruta3 = "Pera";
```



```
let frutas = ["Manzana",
              "Naranja",
              "Pera"];
```

Agrupar datos relacionados en una sola estructura hace que nuestro código sea escalable.

NotebookLM

¿Qué es un Array?

Una lista ordenada donde cada elemento tiene una posición exacta.

¡En JavaScript, siempre empezamos a contar desde cero!

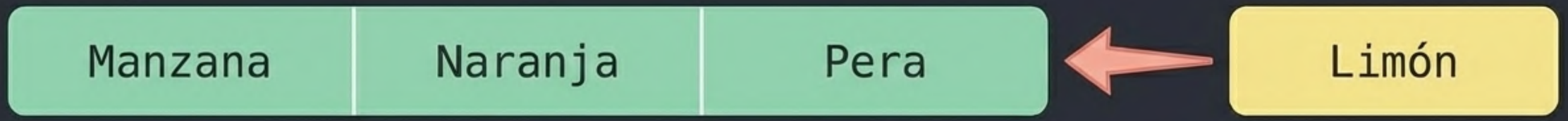
```
let frutas = ["Manzana", "Naranja", "Pera"];
console.log(frutas[0]); // Resultado: "Manzana"
console.log(frutas[1]); // Resultado: "Naranja"
```

NotebookLM

Modificando el almacén: push y pop

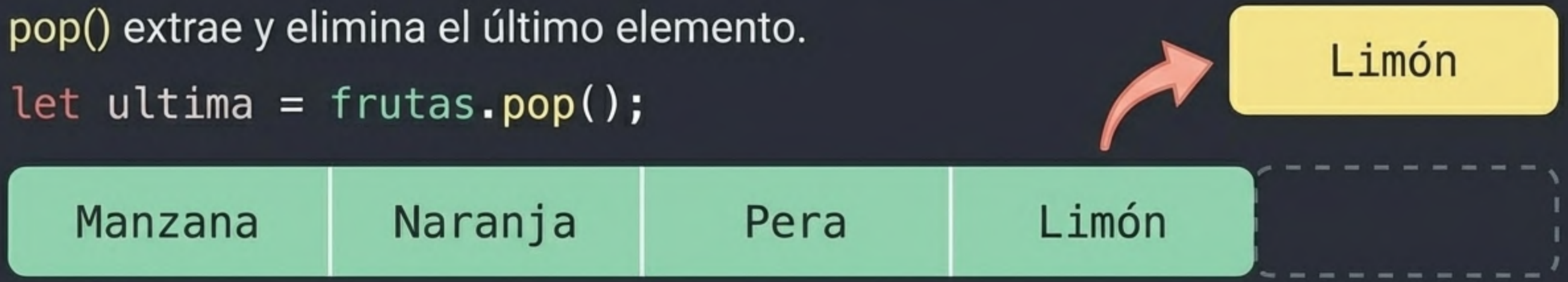
`push()` añade un elemento al final.

```
frutas.push("Limón");
```



`pop()` extrae y elimina el último elemento.

```
let ultima = frutas.pop();
```



NotebookLM

El problema de la repetición

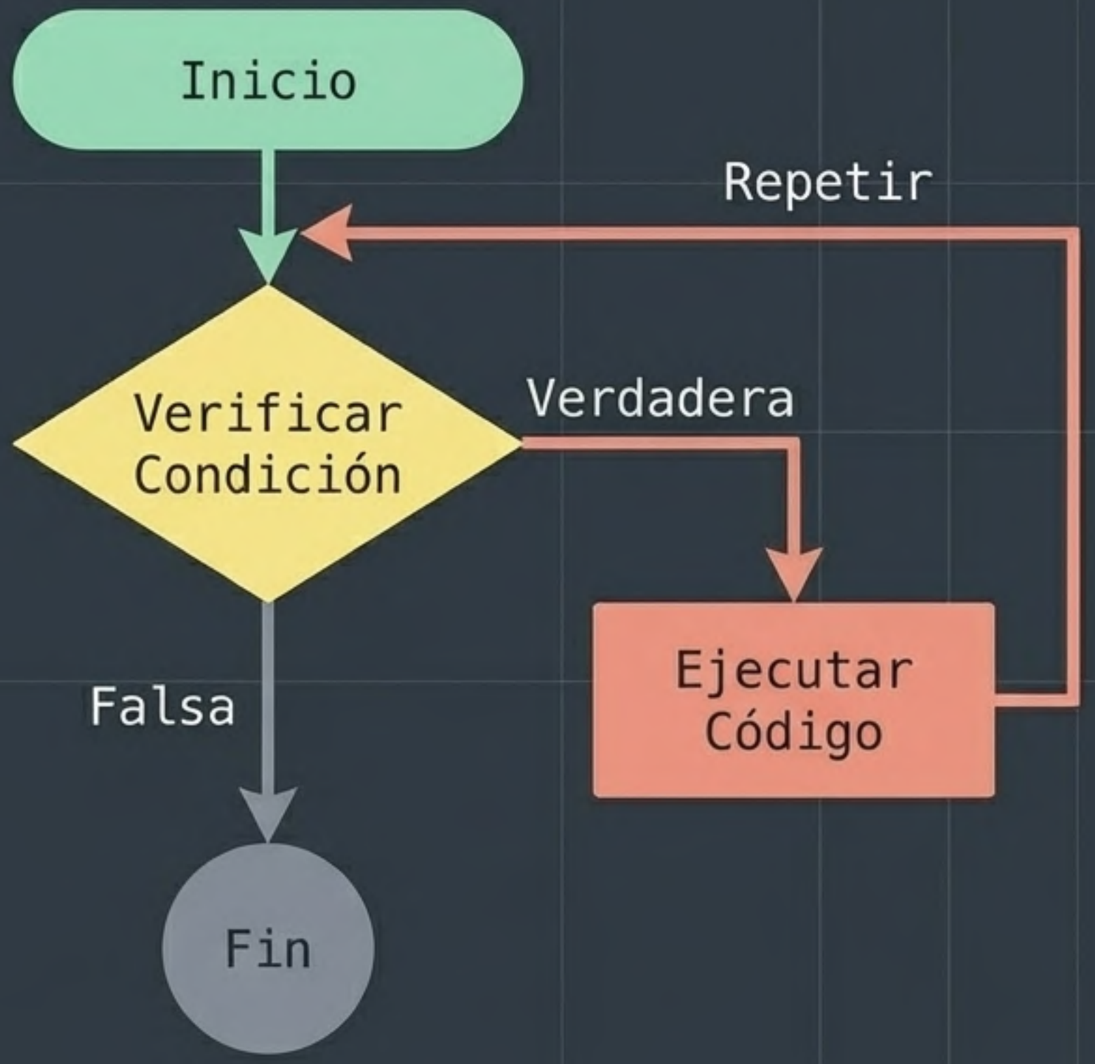
```
console.log(frutas[0]);
console.log(frutas[1]);
console.log(frutas[2]);
console.log(frutas[3]);
...
console.log(frutas[0]);
console.log(frutas[1]);
console.log(frutas[2]);
console.log(frutas[3]);
console.log(frutas[3]);
...
// ... ¿Qué pasa si tenemos
1000 frutas?
```

Los programadores odiamos repetir trabajo manual. Si tienes que escribir lo mismo más de dos veces, necesitas una máquina que lo haga por ti.

NotebookLM

La solución: Automatización mediante bucles

Un `bucle` ejecuta un bloque de código continuamente mientras una **condición** específica siga siendo verdadera.



NotebookLM

El motor básico: El bucle ‘while’

Mientras la condición sea verdadera, sigue ejecutando.

```
let i = 0;
while (i < 3) {
  console.log(i);
  i++;
}
```

1. Contador inicial
2. Condición
3. Acción
4. Paso (incremento)

`i++` es lo mismo que `i = i + 1`. ¡No lo olvides o crearás un bucle infinito!

NotebookLM

La línea de ensamblaje: El bucle 'for'

El bucle for agrupa el inicio, la condición y el paso en una sola línea elegante.

Inicio

```
let i = 0;
```

Condición

```
i < 3;
```

Paso

```
i++
```

```
for (let i = 0; i < 3; i++) { ... }
```

NotebookLM

Anatomía de un bucle 'for'

1. Inicio:

Se ejecuta una sola vez al entrar al bucle.

2. Condición:

Se verifica antes de cada iteración. Si es falsa, el bucle se detiene.

```
for (let i = 0; i < 3; i++) {
    alert(i); }
```

3. Acción:

El código que queremos repetir.

4. Paso:

Se ejecuta después de la acción, preparando la siguiente vuelta.

NotebookLM

El combo definitivo: Recorriendo Arrays

Usamos el contador del bucle como índice del array.

```
let frutas = ["Manzana", "Naranja", "Pera"];
// frutas.length equivale a 3
for (let i = 0; i < frutas.length; i++) {
  console.log(frutas[i]);
}
```

NotebookLM

La forma moderna: “for..of”

JavaScript nos ofrece una sintaxis mucho más limpia cuando solo queremos procesar todos los elementos de un array, sin preocuparnos por el índice numérico.

El método clásico

```
for (let i = 0; i < frutas.length; i++) {
  console.log(frutas[i]);
}
```

La sintaxis moderna

Recomendado para Arrays

```
for (let fruta of frutas) {
  console.log(fruta);
}
```

NotebookLM

Matriz de Decisión: ¿Qué bucle elegir?

Tipo de Bucle	¿Cuándo usarlo?	Complejidad de Sintaxis	¿Ideal para Arrays?
`while`	Cuando no sabes cuántas veces se repetirá la acción.	Media	No recomendado ❌
`for`	Cuando sabes exactamente el número de repeticiones o necesitas el índice matemático.	Alta	Sí (si necesitas el índice) ✅
`for..of`	Cuando quieres leer todos los elementos de una lista, del primero al último.	Baja	Excelente ✅

NotebookLM

Resumen / Cheatsheet

Toma una captura de pantalla de esta guía rápida.

Sección 1: Arrays

Crear:
```
let arr = [1, 2, 3];
```

Acceder: `arr[0]` (primer elemento)

Añadir al final: `arr.push(4);`

Quitar del final: `arr.pop();`

Sección 2: Bucles

Bucle while:
```
while (condicion) { ... }
```

Bucle for:
```
for (let i = 0; i < max; i++) { ... }
```

Recorrer Array moderno:
```
for (let item of arr) { ... }
```

NotebookLM